



Universidad de
Castilla-La Mancha

CAMPUS DE EXCELENCIA INTERNACIONAL



Práctica 2. Conexión y programación de un display con el NodeMCU.

Profesor: Juan Moreno García

Índice

1. Programa ejemplo.
2. Programa a realizar.
3. Mejoras.

1. Programa ejemplo.

Enunciado: Realizar un programa que conecte un display al NodeMCU y permita mostrar en él los números del 0 al 9 leídos desde el *Monitor Serie*, si recibe -1 borrará el contenido del display. Para esto se mostrará en el Monitor Serie el siguiente mensaje

Dame un valor para el display entre 0 y 9, -1 para borrar
después se leerá el valor seleccionado.

Material: El hardware necesario es el siguiente:

- * 1 Placa nodeMCU.
- * 2 Placas de pruebas (Breadboard).
- * 1 display, el programa ha utilizado el modelo TOS8102AE.
- * 8 resistencias de 220 ohms (se pueden omitir, con valores superiores a 220 ohmios deja de verse debido a una alta resistencia).
- * 11 Cables macho a macho.

1. Programa ejemplo.

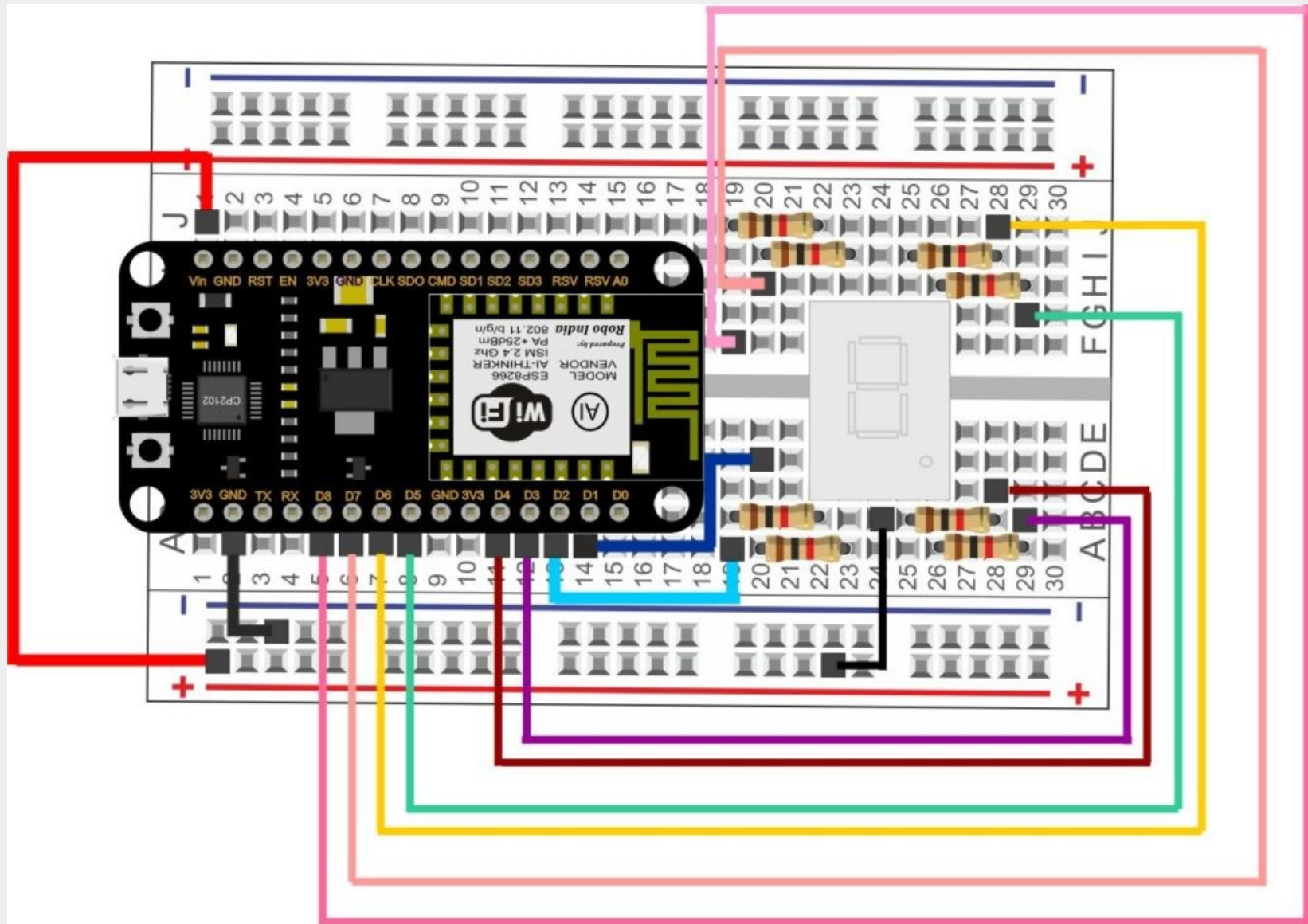


Imagen tomada de: <https://roboindia.com/tutorials/seven-segment-nodemcu/>

1. Programa ejemplo.

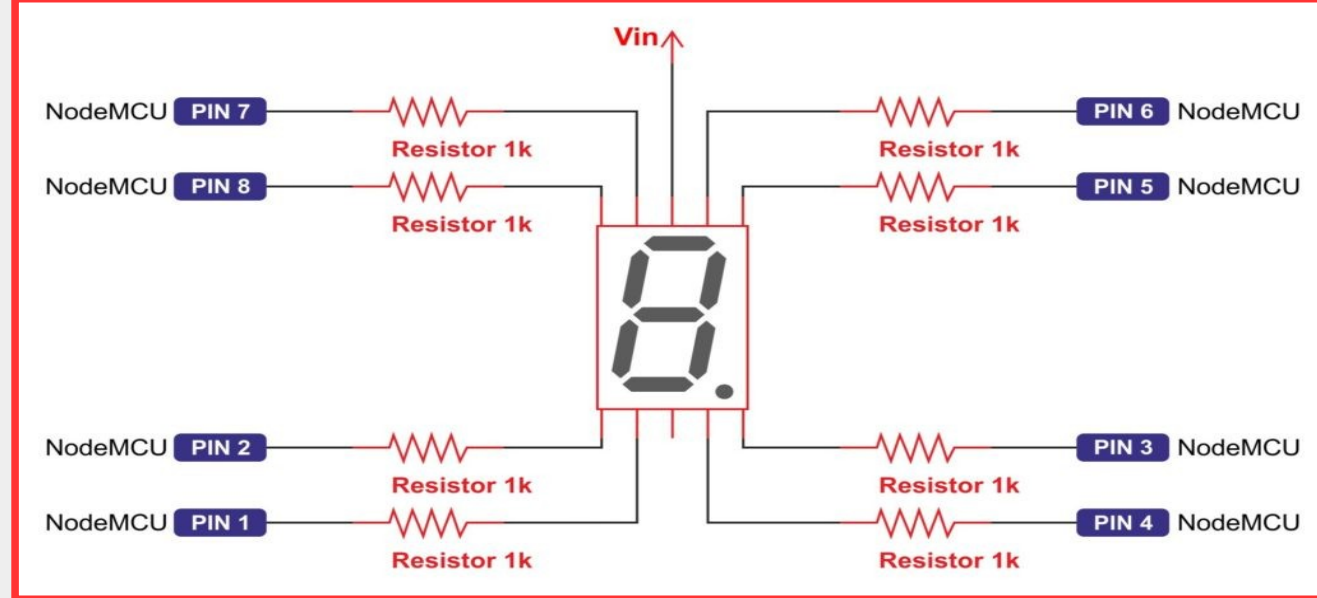
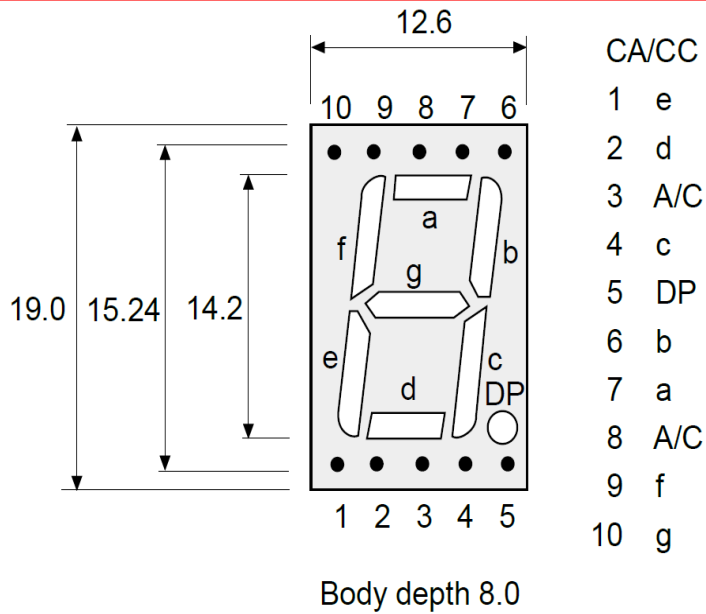
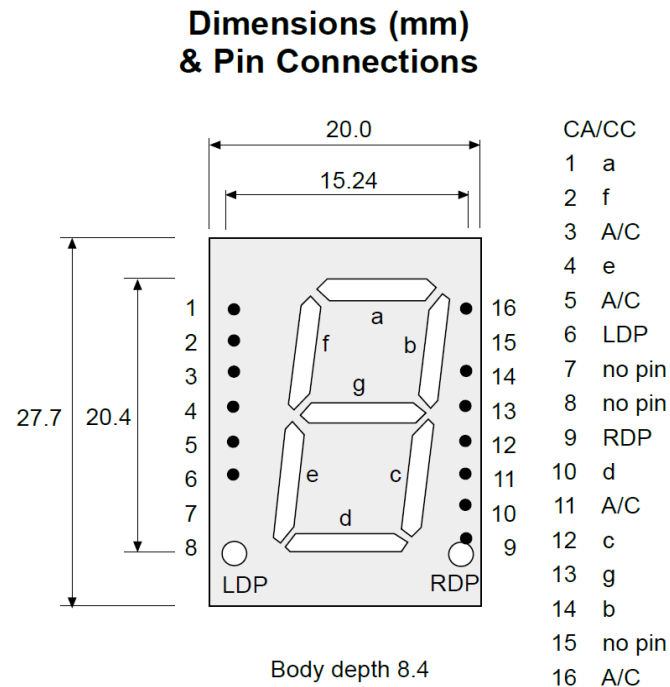


Imagen tomada de: <https://roboindia.com/tutorials/seven-segment-nodemcu/>

Cada **pin del NodeMCU se conecta a un terminal de una resistencia y el otro terminal se conecta al pin seleccionado del display**, concretamente:

- * D6 (GPIO12) ↔ a
- * D5 (GPIO14) ↔ b
- * D4 (GPIO2) ↔ c
- * D1 (GPIO5) ↔ d
- * D2 (GPIO4) ↔ e
- * D7 (GPIO13) ↔ f
- * D8 (GPIO15) ↔ g

Además, se debe **conectar Vin y masa**.



Programa Versión 1

Cabecera y funciones

```
// TOS8102AE y TOS5161AS
const int A = 12; // D6
const int B = 14; // D5
const int C = 2; // D4
const int D = 5; // D1
const int E = 4; // D2
const int F = 13; // D7
const int G = 15; // D8
const int DP = 0; // el punto, de momento no se usa

void setup() {
  pinMode(A, OUTPUT); // Asignación de las salidas digitales
  pinMode(B, OUTPUT);
  pinMode(C, OUTPUT);
  pinMode(D, OUTPUT);
  pinMode(E, OUTPUT);
  pinMode(F, OUTPUT);
  pinMode(G, OUTPUT);
  Serial.begin(115200);
}

String leerLinea() {
  // Espera hasta que haya algo que leer
  while(Serial.available() < 1) {
    delay(1);
  }
  return Serial.readStringUntil('\n'); // Recibe info hasta '\n'
}

void display (int a, int b, int c, int d, int e, int f, int g) {
  digitalWrite(A,a); //Se reciben 7 variables y se asignan
  digitalWrite(B,b); //a cada una de las salidas
  digitalWrite(C,c);
  digitalWrite(D,d);
  digitalWrite(E,e);
  digitalWrite(F,f);
  digitalWrite(G,g);
}
```

Programa Versión 1

Función principal

```
// Dependiendo de cada dígito, se envía a la función display
// los estados (0 y 1) a cada uno de los segmentos
void loop() {
    String r = "";
    r = leerLinea();
    Serial.println(r);
    if (r[0]=='0') {
        display (1,1,1,1,1,0); //escribe 0
    } else if (r[0]=='1') {
        display (0,1,1,0,0,0); //escribe 1
    } else if (r[0]=='2') {
        display (1,1,0,1,1,0); //escribe 2
    } else if (r[0]=='3') {
        display (1,1,1,1,0,0); //escribe 3
    } else if (r[0]=='4') {
        display (0,1,1,0,0,1); //escribe 4
    } else if (r[0]=='5') {
        display (1,0,1,1,0,1); //escribe 5
    } else if (r[0]=='6') {
        display (1,0,1,1,1,1); //escribe 6
    } else if (r[0]=='7') {
        display (1,1,1,0,0,0); //escribe 7
    } else if (r[0]=='8') {
        display (1,1,1,1,1,1); //escribe 8
    } else if (r[0]=='9') {
        display (1,1,1,0,0,1); //escribe 9
    } else {
        display (0,0,0,0,0,0); //borra display
    }
}
```

1. Programa ejemplo.

Programa Versión 2

Mejoras para solucionar problemas de la versión 1:

1. Uso de **vector** para manejar los pines utilizados.
Evita el **excesivo uso de variables enteras y la complejidad** que esto provoca en la versión 1.
2. Uso de **matriz** para manejar los códigos a mostrar por el display. **Facilita el uso de los códigos en el programa y el paso de parámetros a la función *display()*.**
3. La función **leerEntero()** comprueba que el valor a **convertir sea un entero**, en caso de que no devuelve el código -32767. **Corrige el fallo de la anterior versión que devolvía 0** cuando no era un número entero el parámetro

Programa Versión 2

Pines, códigos para el encendido y *setup()*

```
// El orden en los índices es el natural, A->0, B->1, .... G->7
int pines[7] = {12, 14, 2, 5, 4, 13, 15};
int codigos[11][7] = {
    {1,1,1,1,1,1,0}, // escribe 0
    {0,1,1,0,0,0,0}, // escribe 1
    {1,1,0,1,1,0,1}, // escribe 2
    {1,1,1,1,0,0,1}, // escribe 4
    {0,1,1,0,0,1,1}, // escribe 4
    {1,0,1,1,0,1,1}, // escribe 5
    {1,0,1,1,1,1,1}, // escribe 6
    {1,1,1,0,0,0,0}, // escribe 7
    {1,1,1,1,1,1,1}, // escribe 8
    {1,1,1,0,0,1,1}, // escribe 9
    {0,0,0,0,0,0,0} // borra el display
};

void setup() {
    int i;
    for(i=0; i<7; i++) {
        pinMode(pines[i], OUTPUT);
    }
    Serial.begin(115200);
}
```

Programa Versión 2

Función principal y función *display()*.

```
void display(int v[7]) {
    int i;
    for(i=0; i<7; i++) {
        digitalWrite(pines[i],v[i]);
    }
}

// Dependiendo de cada dígito, se envía a la función display
// los estados (0 y 1) a cada uno de los segmentos

void loop() {
    int n;
    n = leerEntero("Dame un valor para el display entre 0 y 9, -1 para borrar");
    Serial.println( String(n) );
    if( (n>=0) && (n<=9) ) {
        display(codigos[n]);
    }
    if (n==-1) {
        display(codigos[10]);
    }
}
```

Programa Versión 2

Funciones para leer entero. Mejorada respecto a las utilizadas hasta este punto.

```
int leerEntero(String s) {
    String r;
    Serial.println(s);
    r = leerLinea();
    if (esEntero(r)) {
        return r.toInt();
    }
    return -32768; // Un código que no usara el loop
}

bool esEntero(String s) {
    int i, p=0;
    if ((s[0]=='+') or (s[0]=='-')) {
        p = 1;
    }
    for(i=p; i<s.length(); i++) {
        if (not isDigit(s[i]) ) {
            Serial.println( "Valor no entero: " + s );
            return false;
        }
    }
    return true;
}

String leerLinea() {
    // Espera hasta que haya algo que leer
    while(Serial.available() < 1) {
        delay(1);
    }
    return Serial.readStringUntil('\n'); // Recibe info hasta '\n'
}
```

2. Programa a realizar.

Enunciado: Realizar un programa que reciba una vocal en minúsculas o mayúsculas (a, e, i, o, u, A, E, I, O, U) y la muestre en el display. El programa también permitirá borrar el display y mostrar una de las 6 últimas letras pulsadas. Para seleccionar la opción mostrará el siguiente menú en el *Monitor Serie*:

Elige una de las siguientes opciones:

a, e, i, o, u, A, E, I, O, U para mostrar ese valor.

b o B para borrar.

v ó V para mostrar uno de los últimos seis valores mostrados.

Se deben utilizar vectores y matrices para manejar el display. Se debe utilizar un vector para contener los últimos 6 valores. Además, se deben utilizar funciones para cada una de las funcionalidades.

Material: El hardware necesario es el siguiente:

- * 1 Placa nodeMCU.
- * 2 Placas de pruebas (Breadboard).
- * 1 display.
- * 8 resistencias de 220 ohms (se pueden omitir, con valores superiores a 220 ohmios deja de verse debido a una alta resistencia).
- * 11 Cables macho a macho.

3. Mejoras.

Añadir la funcionalidad de utilizar el '.' que aparece en el display, por ejemplo, para que cuando se lea una letra mayúscula se encienda el punto y en el caso de una minúscula no.